Lecture 11

- UDP and TCP Demultiplexing
- UDP protocol: features and segment format

Connectionless (i.e., UDP) demultiplexing

Recall:

- How to define a UDP client socket

```
client_socket = socket(AF_INET,
                       SOCK_DGRAM)
```

- When a UDP socket is created in this manner, the transport layer automatically assigns a port number in the range 1024 to 65535 (*0 to 1023: reserved*)
- We can also force a specific port number (say, 19157)

```
client_socket.bind(('', 19157))
```

- when creating datagram to ***send*** into UDP socket, we must specify
  - destination IP address
  - destination port #

```
client_socket.sendto(message.encode(),
                     server_address)
```

Connectionless (i.e., UDP) demultiplexing

When receiving host ***receives*** *UDP* segment:

- checks destination port # in segment
- directs UDP segment to socket with that port #



UDP datagrams with *same destination port #,* but different source IP addresses and/or source port numbers will be directed to the *same socket* at receiving host

UDP Demultiplexing

Example:

- server_socket.bind((`’, 12000))

IP address **Port number**

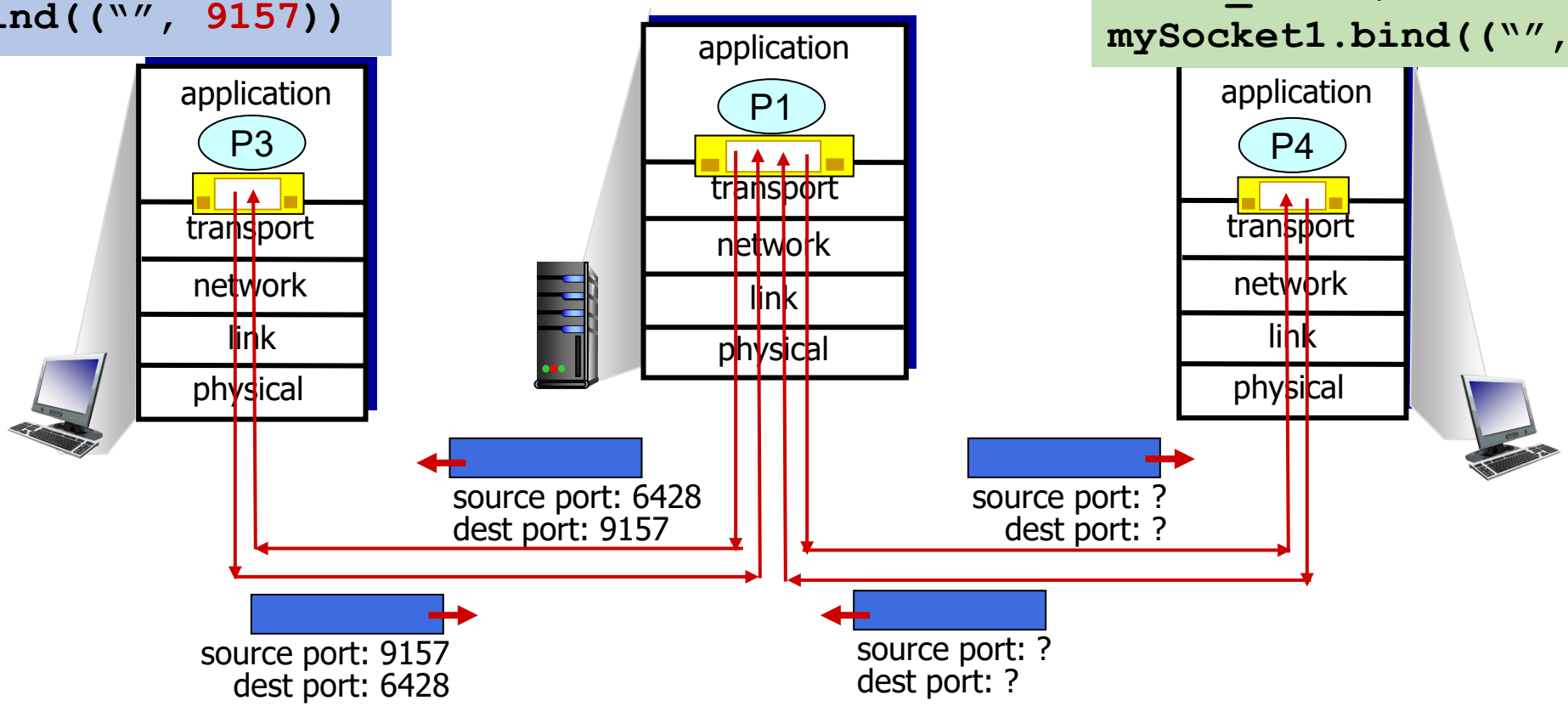
- Client A: 10.0.0.1:5000 → Server: 192.168.1.10:12000
- Client B: 10.0.0.2:6000 → Server: 192.168.1.10:12000
- Client C: 10.0.0.3:7000 → Server: 192.168.1.10:12000

Connectionless demultiplexing: an example

```
serverSocket =
  socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(("",6428));
```

```
mySocket2 = socket(AF_INET,
                   SOCK_DGRAM)
mySocket2.bind(("", 9157))
```

```
mySocket1 = socket(AF_INET,
  SOCK_DGRAM)
mySocket1.bind(("",5775))
```



UDP Demultiplexing

Example:

- server_socket.bind((`’, 12000))

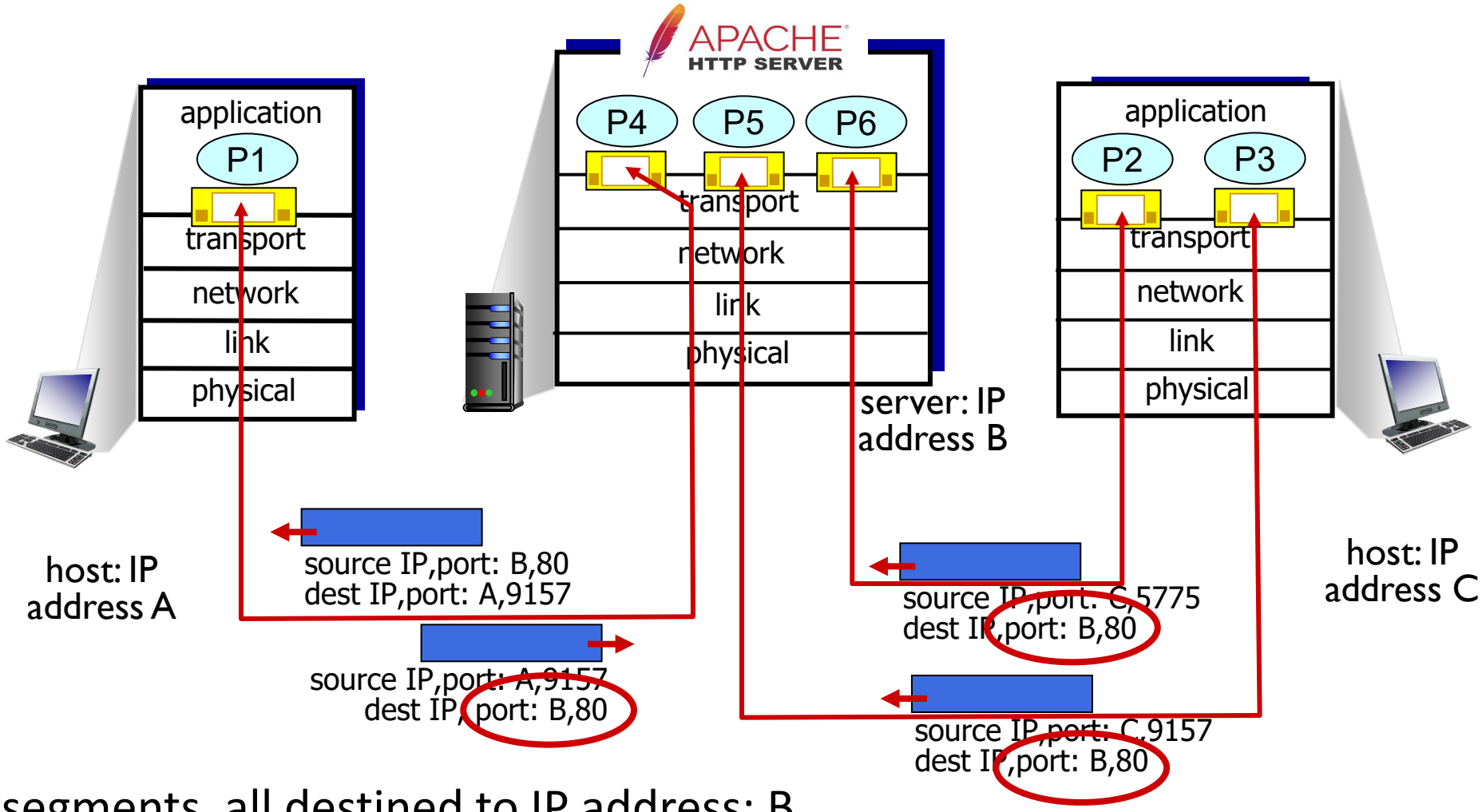
IP address **Port number**

- Client A: 10.0.0.1:5000 ➔ Server: 192.168.1.10:12000
- Client B: 10.0.0.2:6000 ➔ Server: 192.168.1.10:12000
- Client C: 10.0.0.3:7000 ➔ Server: 192.168.1.10:12000

How does the server distinguish data from different clients/processes?

- message, client_address = server_socket.recvfrom(2048)
- server_socket.sendto(response, client_address)

Connection-oriented demultiplexing: example



Three segments, all destined to IP address: B,
Destination port: 80 are demultiplexed to *different* sockets

Connection-oriented demultiplexing

- TCP socket identified by 4-tuple:
  1) source IP address
  2) source port number
  3) dest IP address
  4) dest port number
- demux: receiver uses *all four values (4-tuple)* to direct segment to appropriate socket

- server may support many simultaneous TCP sockets:
  - each socket identified by its own 4-tuple
  - each socket associated with a different connecting client

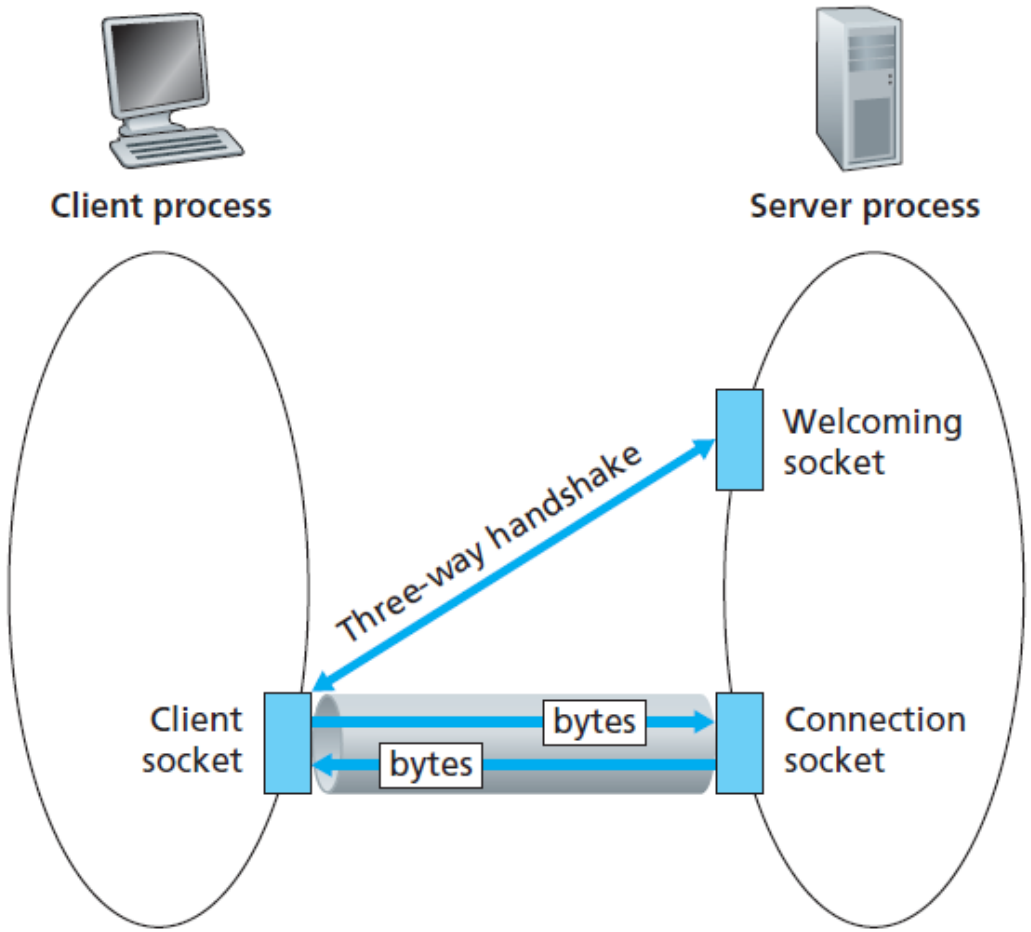
Summary

- **UDP:** demultiplexing using destination port number
- **TCP:** demultiplexing using 4-tuple: source and destination IP addresses, and port numbers

Question: TCP Sockets

- Suppose 100 clients are simultaneously communicating with a (traditional HTTP/TCP) web server. How many sockets are respectively at the server and at each client?
  A. 1,1
  B. 2,1
  C. 200,2
  D. 100,1
  E. 101, 1

Welcoming Socket



Q3: UDP Sockets

- Suppose we use UDP instead of TCP under HTTP for designing a web server where all requests and responses fit in a single packet. Suppose 100 clients are simultaneously communicating with this web server. How many sockets are respectively at the server and at each client?
  A. 1,1
  B. 2,1
  C. 200,2
  D. 100,1
  E. 101, 1

Question: Sockets and multiplexing

Select those statements that are **TRUE**.

a) Compared with UDP, TCP uses more information in packet headers to demultiplex packets.

b) Transport-layer multiplexing allows multiple application processes on the same host to send data over the network simultaneously.

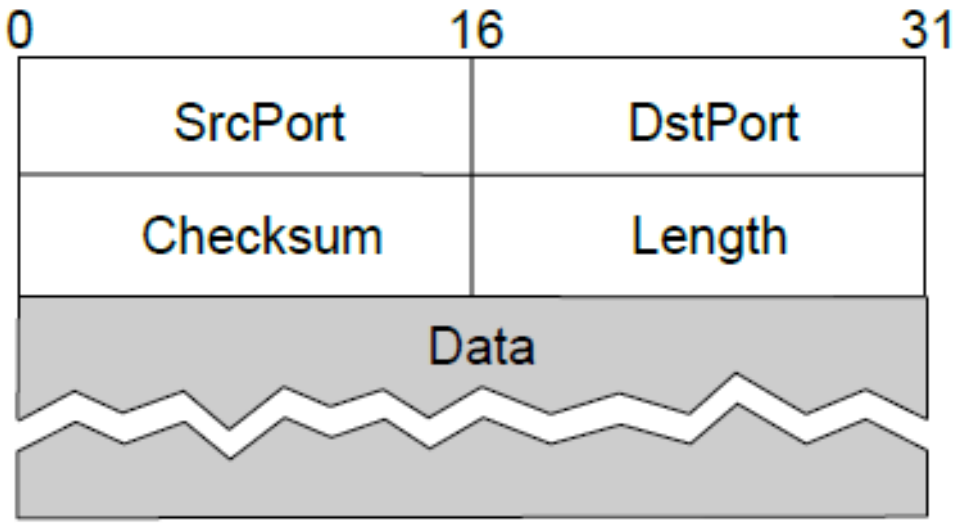
c) If two TCP clients connect to the same web server on port 80, the server creates a separate socket for each connection. But both sockets will have port number 80.

d) For UDP, all segments sent to the same destination port number are delivered to the same socket, regardless of the sender.

Connectionless transport: UDP

UDP: User Datagram Protocol

- *Connectionless:*
  - No handshaking between UDP sender, receiver
  - Each UDP segment handled independently of others
- Unreliable and unordered datagram service
- Header format:



UDP: User Datagram Protocol

Why is there a UDP?

- No connection establishment (which can add RTT delay)
- simple: no connection state at sender, receiver
- small header size (8 bytes vs. 20 bytes for TCP)
- no congestion control
  - UDP can blast away as fast as desired!
  - can function in the face of congestion

UDP: User Datagram Protocol

- Use cases that ***primarily*** use UDP:
  - DNS
  - SNMP (Simple Network Management Protocol)
  - Streaming multimedia apps (loss tolerant, rate sensitive): TCP vs. UDP choice?
- If reliable transfer needed over UDP: Need to implement reliability features at the application layer. Any example?

UDP: User Datagram Protocol [RFC 768]

INTERNET STANDARD

RFC 768

J. Postel
ISI
28 August 1980

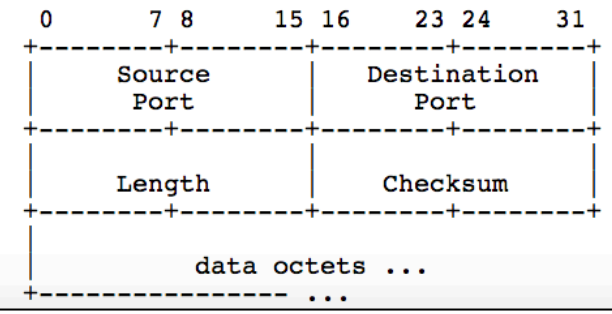
User Datagram Protocol

Introduction

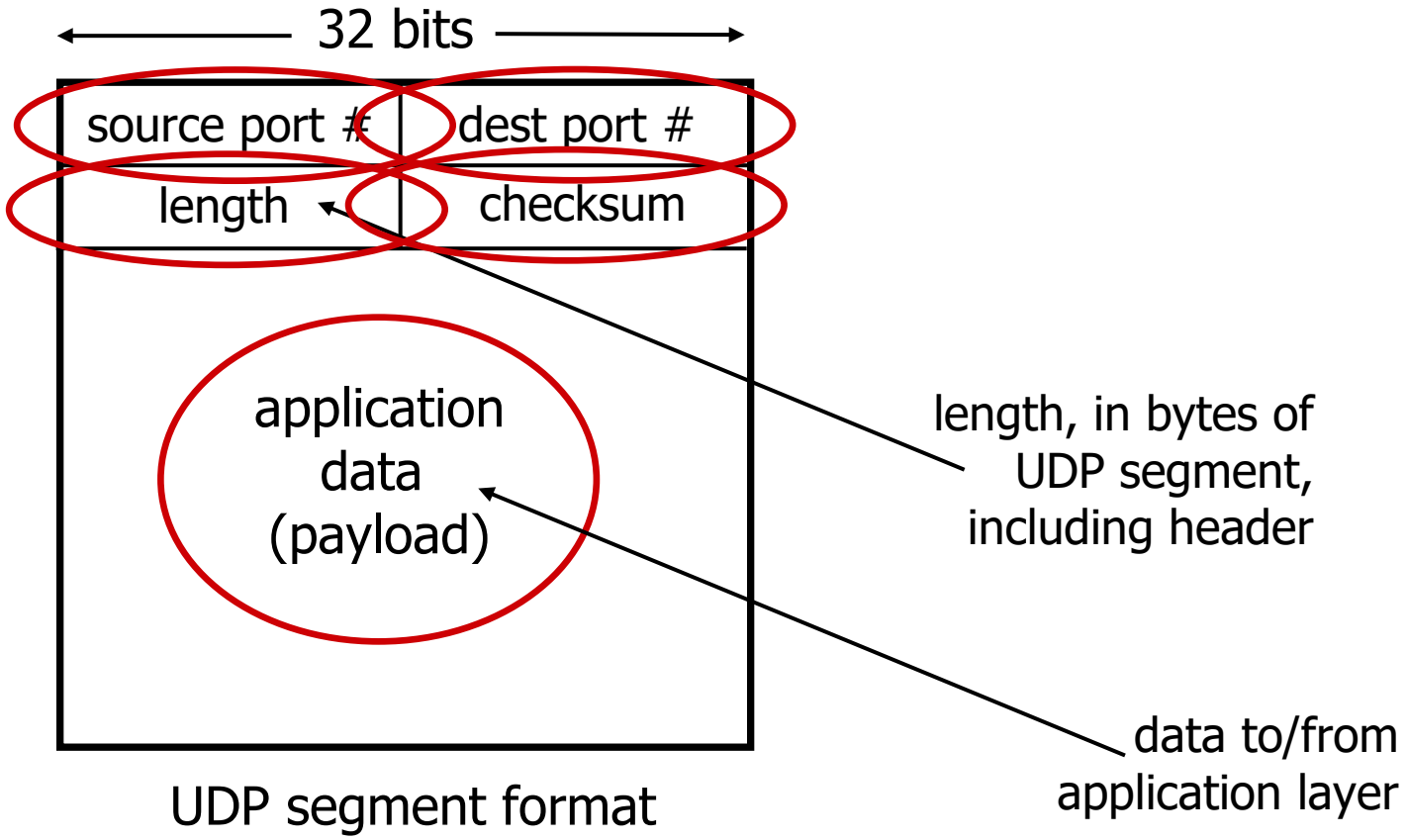
This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

Format



UDP segment header



How do we know destination port #?

- Either well-known port numbers
- DNS: 53, SNMP: 161 or 162
- Or we get this information through other protocols before sending UDP segments